AI, SOFTWARE DELIVERY, AND THE TEAM

Are Traditional Software Teams a Thing of the Past?

How AI changes the work, the workflow, and the shape of effective teams.

THE PROBLEM / THE OPPORTUNITY

The software engineering process is broken.

The good news?

PEOPLE AND TEAMS

You just got promoted.



Captain Phillips / Know Your Meme

AUDIENCE QUESTION

What are we now?

PMs? CEOs? Super Humans? Useless?

YOU HAVE ACCESS TO LABOR

Your output is the team's output.

You are the manager now

Labor	Agents on demand, at any hour.
Process	How people and agents work together.
Output	What the whole team ships, verified.

Same app. New tools.

Beam, 2018	BeamRS, 2026
Blazor WebAssembly	Rust + Leptos, server-rendered
ASP.NET Core API	Axum API
SQL Server	PostgreSQL + Docker

The fully agentic engineering rewrite

BEAMRS

The app worked. It just existed twice.

A good-looking demo is not proof of a correct system.

THE THESIS

The bottleneck moved from writing code to trusting it.

Plausible isn't Proven

THE BOTTLENECK MOVED

If code took no time, what would still stop your team shipping?

A THREE-YEAR DEAD END / BEFORE AI

The Three-Year Rewrite

A great deal of work

Several teams.
More than three years.
A finished application.

No viable outcome

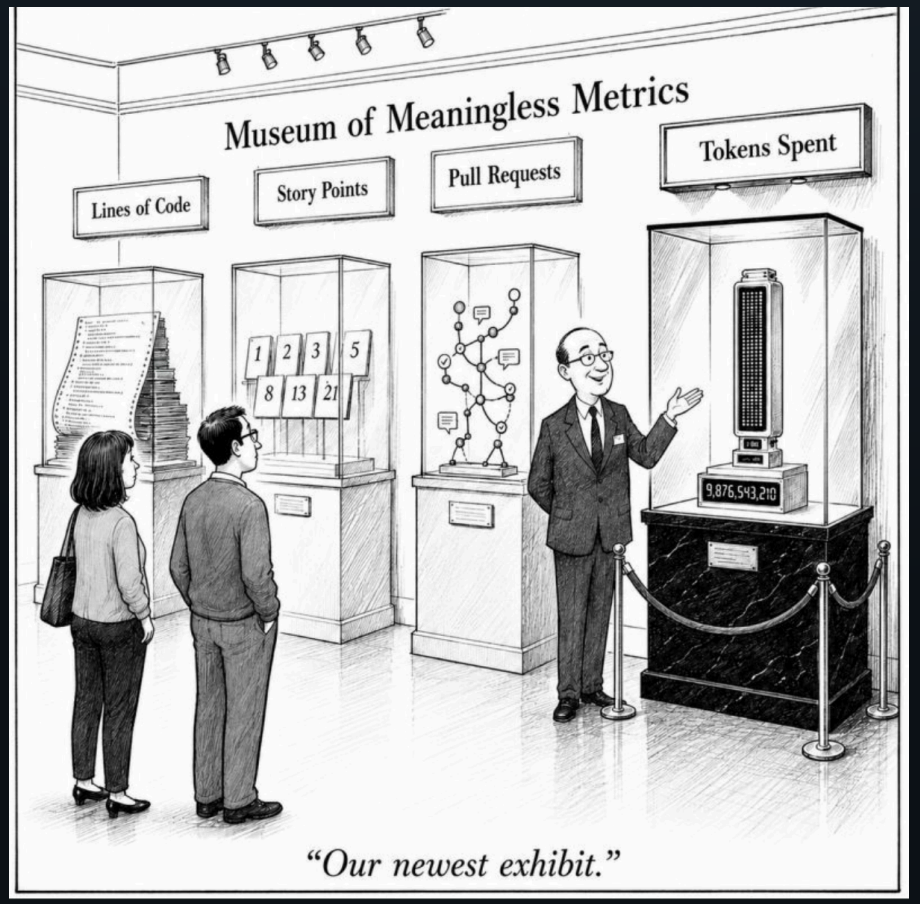
It cost more to run than
it could ever earn.
It was scrapped.

They didn't have a speed problem.

VALUE, NOT VOLUME

Output isn't outcome.

Tasks closed. Pull requests merged. Code generated.
None tells you whether it was worth doing.



"Museum of Meaningless Metrics" · Gamble Blog

THE REWRITE, REVISITED

Four questions to answer first.

ASK UP FRONT	THE THREE-YEAR REWRITE
Value: What will it earn or save?	It could never pay for itself
Economics: What will it cost to run?	It cost more to run than it earned
Thin slice: What reaches users early?	Three-plus years to a finished app
Stop rule: What result stops us?	Scrapped only after it was done

With agents, the same dead end arrives faster.

FIRST AGENT WORKFLOWS / RPI → QRSPI → QRSPI+

Building with AI, Good prompts are not enough

Instruction overload

Important guidance gets lost.

Magic words

Behavior depends on knowing what to say.

Persuasive plans

Readable does not mean correct.

CASE STUDY A / THE RELAY

A Relay Workflow

Ideator → Planner → Coder → Tester

AI Grades itself

Required coverage

>90%

Reported evidence

83%

Written handoffs shipped features.
The evidence said otherwise.

AN EVOLUTION OF WORKFLOWS

RPI → QRSPI → QRSPI+

RPI

Research → Plan → Implement

Understand the code before writing it.

QRSPI

Questions → Research → Design → Structure → Plan

↵ Work tree → Implement → Pull request

QRSPI+

Goals → Questions → Research → Design → Phasing → Structure → Plan

↵ Parallelize → Implement → Integrate → Test → PR

Simplified main paths; QRSPI+ also has a quick-fix route.

CASE STUDY B / QRSPI+ / 2026

A workflow built its next release.

38	20	2,083
Tasks	Waves of work	Passing tests

Scale is context. The important part is what slipped.

CASE STUDY B / WHAT THE GATES CAUGHT

A safeguard vanished in a merge.

Wrong tasks merged → **Integration missed it** → **Acceptance caught it**

The fix restored the safeguard.
The next release added a merge-content check.

WHAT EVOLVED

From confidence to evidence.

The Relay	QRSPI+
The builder grades itself.	Independent checks, then a person.
Instructions keep growing.	Focused steps, measured budget.
A report says it passed.	A miss becomes a new check.

WHAT'S NEXT

QRSPI+ → what's next

QRSPI+ A fixed route of up to 12 steps, heavy on ceremony.

Next

Goals → Adaptive build → Evaluate → Review

Keep what wins.
Better results → new baseline

Undo the rest.
No improvement → restore baseline

Reduced complexity

Only the steps the task needs.

Evaluation

Measured against a baseline.

Review

A person decides what ships.

EVALUATE THE PROCESS

Prove the new process is better.

Past tasks → Baseline run → Changed run → Compare outcomes

Same tasks

Real past work, not demos.

Many runs

One good run can be luck.

Outcomes

Acceptance, defects, rework, cost.

AVOID OVERFITTING

Overfitting the process to today's problems.

The risk	The guard
Every incident adds a rule.	Retire steps that stop catching.
Built around today's model.	Re-test when models change.
Tuned to familiar tasks.	Hold out tasks you never tune on.

Choosing a tool isn't using it.

Stack	Rust UI chosen. Agent wrote 227 lines of JavaScript.
Toolchain	A clean Docker build needed a newer Rust.
Load testing	Built in-house. Real HTTP added when it mattered.
Process	Same 12-step route for a rewrite and a version bump.

BEAMRS / LEPTOS UPGRADE / PULL REQUEST REVIEW

Working locally wasn't enough.

Duplicated UI	Attach to the server-rendered page.
Lost settings	Preserve runtime deployment settings.
Repeated requests	Make one API call per page load.
Missing process	Record integration and replanning.

Success had hidden failure modes.

Memory	A setting could consume enormous memory.
False success	An incomplete response could count as passing.
Cleanup	Workers could outlive a crash or cancellation.

All three fixes added regression tests.

CLOSE THE LOOP

Every finding should make the next run harder to fool.

Finding → Fix + reply → Regression test → Better process

ILLUSTRATIVE USE CASE

The repository becomes a workplace.

Event or schedule → **Bounded agent** → **Visible evidence**

This could be GitHub Agentic Workflows.

GITHUB AGENTIC WORKFLOWS / GOVERNED AUTOMATION

Define authority before execution.

Context

A defined trigger.
A focused task.

Permissions

Read-only by default.
Approved write actions.

Evidence

Issues, comments,
pull requests.

NATURAL LANGUAGE UNCERTAINTY

Imprecise instructions. Unpredictable execution.

English is the programming language now.

NATURAL LANGUAGE UNCERTAINTY

It's natural language all the way down.

Harness	A system prompt and tool descriptions.
Skill	A Markdown file of instructions.
Agent	A role described in prose.
Your prompt	One more paragraph on top.

More layers add more interpretation, not more precision.

NATURAL LANGUAGE UNCERTAINTY / AN ENGLISH EXAMPLE

“I saw the man with the telescope.”

READING 1

I used the telescope to see him.

READING 2

He was the one holding the telescope.

IMPLIED

He stole my telescope.

IMPLIED

I already found someone I was looking for.

All are plausible. An agent picks one.

NATURAL LANGUAGE UNCERTAINTY

Pin intent down with something precise.

Acceptance tests	Done is something that runs.
Worked examples	Show the reading you meant.
Schemas	Structure a script can check.
Scripts	The same steps every run.

The workflow is software. Engineer it like software.

Evaluate	Fixed tasks, a baseline, outcome measures.
Limit	Budgets for tokens, time, retries, and cost.
Hand off	Structured artifacts a script can check.

GUARD AGAINST OURSELVES

Judge the process by evidence, not by favorites.

Confirmation bias

We keep what feels better.
We drop what failed once.

Trim the fat

Every step has a cost.
Cut what never catches anything.

GITHUB'S AGENTIC ENGINEERING SYSTEM

A framework that might help

	AES	IN THIS TALK
Stocks	Governance, knowledge, value	Checks, context, outcomes
Activities	Define, deliver, detect	Goals, build, evaluate & review
Modes	Director, performer, assessor	People or agents, set by risk

CAREERS AND TEAMS

Responsibilities change. Accountability stays.

Product & domain	Intent, constraints, value, success criteria.
Engineering	Architecture, decomposition, integration.
Developers	Implement, delegate, debug, inspect.

TEAM DESIGN

More agents need more decision capacity.

Quality	Design evidence that can disprove success.
Platform	Provide context, workflows, permissions.
Management	Balance agent capacity with human review.

AUDIENCE EXERCISE

Add agents to your org chart. Where do review arrows go?

THE PROCESS IS A PRODUCT

This talk used the same loop.

Intent	Start with the abstract and audience.
Research	Check claims against original evidence.
Review	Separate passes for flow and messaging.
Human correction	Decide what is true and what matters.

MEASURE OUTCOMES, NOT ACTIVITY

Did value reach users?

User and business outcomes
Earned or saved value
Cost to run
Validated delivery time
Review, rework, late defects

≠

Activity signals
Tasks closed
Pull requests merged
Story points
Lines generated

RETURN TO THE OPENING

How would your team have known?

That it was correct.
That it was worth building.

THE ANSWER

Traditional teams aren't a thing of the past. Teams that only write code are.

ACKNOWLEDGMENTS

Special thanks

Daniel Frysinger

For his work on QRSPI+ and agent workflows.

Trash Pandas team

For their feedback.

Evan Boyle

For written research on adaptive agent delivery.

Alex Lavaee

For “From RPI to QRSPI” and its research on evolving agent workflows.

DISCUSSION

Questions?

Plausible isn't proven.

APPENDIX / SOURCE TRAIL

Follow the evidence.

Talk outline & research links

Complete source list and case caveats

BeamRS: load-testing review

Memory, false success, worker cleanup

From RPI to QRSPI

Why the workflow evolved

BeamRS: upgrade review

Duplicated UI, settings, requests, process

QRSPI+: release run

Tasks, gates, and acceptance evidence

GitHub Agentic Workflows

Repository automation and authority